Dockerizing a FastAPI RAG Application

Complete Step-by-Step Manual for Windows & macOS

SimpleRAG Deployment Guide · 2026

What this manual covers

- Verify Docker is correctly installed and running
- Clone and inspect the SimpleRAG FastAPI project
- Write a Dockerfile from scratch
- Build and tag a Docker image
- Run the container with and without environment variables
- Manage containers with essential Docker commands

Platforms covered: Windows 10/11 (PowerShell) and macOS (Terminal / Zsh/Bash)

Introduction

This manual walks you through the complete process of containerizing the SimpleRAG FastAPI application using Docker. Docker packages your application and all its dependencies into a portable container that runs identically on any machine — no more "it works on my machine" problems.

Why Docker?

- Consistent environment across development, staging, and production
- Easy deployment — one image runs anywhere Docker is installed
- Isolated dependencies — no conflicts with other Python projects
- Simple to share and reproduce your exact setup

Prerequisites

Before starting, make sure the following are installed on your machine:

Requirement	Windows	macOS
Docker Desktop	`docker.com/get-started`	`docker.com/get-started`
Git	`git-scm.com/download/win`	`brew install git`
Text Editor	VS Code or Notepad++	VS Code or nano

Phase 1: Verify Docker Installation

Before anything else, confirm Docker is properly installed and the Docker daemon (background service) is running. If Docker is not running, none of the subsequent steps will work.

Step 1.1 — Check Docker Version

Open your terminal and run:

Windows (PowerShell / CMD)	macOS (Terminal)
`docker --version`	`docker --version`

INFO: Expected output: Docker version 24.x.x or higher. If you get 'command not found', Docker Desktop is not installed or not added to PATH.

Step 1.2 — Verify the Docker Daemon is Running

This command checks whether the Docker Engine (the server component) is active and responsive:

```
docker info
```

You should see two sections of output — Client and Server information. The Server section confirms the daemon is running.

NOTE: If you only see Client info and an error for Server, open Docker Desktop and wait for it to fully start (the whale icon in the system tray should stop animating).

Step 1.3 — Run the Hello World Container

This is the classic Docker smoke test. It downloads a tiny test image, runs it as a container, prints a message, and exits:

```
docker run hello-world
```

Expected output includes the line:

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

TIP: This command pulls the hello-world image from Docker Hub. If it fails with a network error, check your internet connection or corporate proxy settings.

Phase 2: Clone the Project Repository

Now that Docker is confirmed working, clone the SimpleRAG project from GitHub. Git will download all source files into a new folder on your machine.

Step 2.1 — Clone the Repository

Windows	macOS

`# Open PowerShell or Git Bash`
`git clone https://github.com/AhmadMahi/`
`  SimpleRAG-Deployment-Demo.git`
	`# Open Terminal`
	`git clone https://github.com/AhmadMahi/`
	`  SimpleRAG-Deployment-Demo.git`

TIP: This creates a folder called SimpleRAG-Deployment-Demo in your current working directory. Make sure you are in a location you prefer (e.g., Desktop or Documents) before running this.

Step 2.2 — Navigate into the Project

```
cd SimpleRAG-Deployment-Demo
```

Step 2.3 — Verify the Project Structure

Windows	macOS
`dir`	`ls`

Expected output:

```
app
README.md
requirements.txt
static
uploads
render.yaml
vercel.json
```

INFO: The app/ directory contains the FastAPI application code. requirements.txt lists all Python dependencies Docker will install.

Step 2.4 — Inspect the Application Entry Point

Confirm the Python files exist inside the app directory:

Windows	macOS
`Get-ChildItem -Recurse -Filter *.py`	`find app -name "*.py"`

You should see files including app/main.py, app/config.py, and app/models.py. The main.py is the FastAPI entry point referenced in the Docker CMD instruction.

Phase 3: Create the Dockerfile

A Dockerfile is a plain text recipe that tells Docker how to build your application image. Each line is a layer — Docker caches layers to speed up subsequent builds.

Step 3.1 — Create the Dockerfile

Windows	macOS

`# In PowerShell:`
`New-Item Dockerfile`
 	
`# Or using Git Bash:`
`touch Dockerfile`
	`touch Dockerfile`

NOTE: Do NOT add any file extension. The file must be named exactly Dockerfile (capital D). Docker will not find it otherwise.

Step 3.2 — Edit the Dockerfile

Windows	macOS

`# Open with VS Code:`
`code Dockerfile`
 	
`# Or Notepad:`
`notepad Dockerfile`
	`# Open with VS Code:`
	`code Dockerfile`
	
	`# Or nano:`
	`nano Dockerfile`

Paste the following content exactly:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Understanding Each Dockerfile Instruction

Instruction	What it does
`FROM python:3.12-slim`	Base image — a minimal Python 3.12 Linux environment. The -slim variant omits dev tools, keeping the image small.
`WORKDIR /app`	Sets the working directory inside the container. All subsequent commands run from /app.
`COPY requirements.txt .`	Copies only requirements.txt first. This allows Docker to cache the pip install layer separately.
`RUN pip install ...`	Installs all Python dependencies listed in requirements.txt. --no-cache-dir keeps image size smaller.
`COPY . .`	Copies the rest of your project files into /app inside the container.
`EXPOSE 8000`	Documents that the container listens on port 8000. Does not actually publish the port — that happens at runtime.
`CMD [...]`	Default command to run when the container starts. Launches the FastAPI app via Uvicorn on all network interfaces.

Step 3.3 — Verify the Dockerfile

Windows	macOS
`Get-Content Dockerfile`	`cat Dockerfile`

NOTE: Double-check that the CMD line uses double quotes around each argument and square brackets. Missing quotes or using single quotes will cause the container to fail.

Phase 4: Build the Docker Image

With the Dockerfile ready, you can now build the image. Docker reads your Dockerfile layer by layer and produces a self-contained image.

Step 4.1 — Build the Image

```
docker build -t simplerag .
```

Breaking down this command:

Part	Meaning
`docker build`	Build a new image from a Dockerfile
`-t simplerag`	Tag (name) the image as simplerag:latest
`.`	Use the current directory as the build context (where Docker looks for the Dockerfile and files to copy)

INFO: The first build takes a few minutes as it downloads the Python base image and installs dependencies. Subsequent builds are much faster thanks to layer caching.

Expected final lines of output:

```
Successfully built abc123def456
Successfully tagged simplerag:latest
```

Step 4.2 — Verify the Image Was Created

```
docker images
```

You should see simplerag listed:

```
REPOSITORY    TAG       IMAGE ID       CREATED          SIZE
simplerag     latest    abc123def456   1 minute ago     245MB
```

NOTE: If the image does not appear, the build likely failed. Scroll up in your terminal to find the error line — it usually points to a missing dependency in requirements.txt or a syntax error in the Dockerfile.

Phase 5: Run the Container

The image is built and ready. Now run it as a container — a live, running instance of your application.

Step 5.1 — Start the Container (Basic)

```
docker run -p 8000:8000 simplerag
```

Flag	Meaning
`-p 8000:8000`	Maps your machine's port 8000 to the container's port 8000 (format: host:container)
`simplerag`	The name of the image to run

Expected output:

```
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Step 5.2 — Open the Application

With the container running, open any web browser and navigate to:

```
http://localhost:8000
```

TIP: You should see the SimpleRAG web interface. The application is fully containerized and running inside Docker.

Step 5.3 — Stop the Container

Windows	ɑ macOS

`# In the same terminal:`
`Ctrl + C`
 	
`# From another terminal:`
`docker ps`
`docker stop <container_id>`
	`# In the same terminal:`
	`Ctrl + C`
	
	`# From another terminal:`
	`docker ps`
	`docker stop <container_id>`

Phase 6: Environment Variables

The SimpleRAG application requires an OpenAI API key to function. It is best practice to never hardcode secrets in your code or Dockerfile. Instead, use a .env file that is loaded at runtime.

Step 6.1 — Create the Environment File

Windows	macOS

`# PowerShell:`
`New-Item .env`
 	
`# Git Bash:`
`touch .env`
	`touch .env`

Step 6.2 — Edit the Environment File

Windows	macOS

`code .env`
`# or`
`notepad .env`
	`nano .env`
	`# or`
	`code .env`

Add the following content (replace the placeholder with your real API key):

```
OPENAI_API_KEY=sk-your-key-here
OPENAI_MODEL=gpt-4o
```

NOTE: Never commit .env to Git. Add .env to your .gitignore file to prevent accidentally sharing your API keys publicly.

Step 6.3 — Run Container with Environment Variables

```
docker run --env-file .env -p 8000:8000 simplerag
```

Flag	Meaning
`--env-file .env`	Load all KEY=VALUE pairs from .env and inject them as environment variables inside the container
`-p 8000:8000`	Port mapping as before
`simplerag`	Image to run

Phase 7: Essential Docker Commands Reference

These are the day-to-day commands you will use most often when working with Docker containers.

Container Management

Command	Description
`docker ps`	List all currently running containers
`docker ps -a`	List all containers including stopped ones
`docker stop <id>`	Gracefully stop a running container
`docker start <id>`	Start a previously stopped container
`docker rm <id>`	Delete a stopped container
`docker logs <id>`	View output/logs from a container
`docker exec -it <id> bash`	Open a bash shell inside a running container

Image Management

Command	Description
`docker images`	List all local images
`docker rmi simplerag`	Delete the simplerag image
`docker rmi $(docker images -q)`	Delete ALL local images (use with caution)
`docker pull python:3.12-slim`	Download an image from Docker Hub
`docker build -t name .`	Build image from current directory Dockerfile
`docker tag simplerag v1.0`	Add a version tag to an image

Running Containers with Common Options

Command	Description
`docker run -d simplerag`	Run in detached (background) mode
`docker run --name myapp simplerag`	Give the container a custom name
`docker run --rm simplerag`	Auto-remove container when it stops
`docker run -e KEY=VALUE simplerag`	Pass a single environment variable
`docker run --env-file .env simplerag`	Load all env vars from a file

Complete Workflow — Quick Reference

Copy and paste this complete sequence to go from nothing to a running container:

```
# 1. Verify Docker
docker --version
docker info
docker run hello-world

# 2. Clone project
git clone https://github.com/AhmadMahi/SimpleRAG-Deployment-Demo.git
cd SimpleRAG-Deployment-Demo

# 3. Create Dockerfile
touch Dockerfile   # or: New-Item Dockerfile  (Windows)
# [edit Dockerfile with content from Phase 3]

# 4. Build image
docker build -t simplerag .
docker images

# 5. Run without env vars (basic test)
docker run -p 8000:8000 simplerag

# 6. Add environment variables
touch .env   # or: New-Item .env  (Windows)
# [add OPENAI_API_KEY and OPENAI_MODEL to .env]

# 7. Run with env vars
docker run --env-file .env -p 8000:8000 simplerag
```

TIP: Open http://localhost:8000 in your browser after step 5 or 7 to confirm the application is running.

Troubleshooting Common Issues

Docker daemon is not running

Symptom: Cannot connect to the Docker daemon at unix:///var/run/docker.sock

Windows Fix	macOS Fix
Open Docker Desktop from the Start menu and wait for it to fully start. The taskbar icon should stop animating.	Open Docker Desktop from Applications or run: open -a Docker

Port 8000 already in use

Symptom: Error starting userland proxy: listen tcp4 0.0.0.0:8000: bind: address already in use

Windows	macOS

`# Use a different host port:`
`docker run -p 8001:8000 simplerag`
 	
`# Or find and kill the process:`
`netstat -ano | findstr :8000`
	`# Use a different host port:`
	`docker run -p 8001:8000 simplerag`
	
	`# Or find and kill the process:`
	`lsof -i :8000`
	`kill -9 <PID>`

Build fails: requirements not found

Symptom: COPY failed: file not found in build context

NOTE: Make sure you are inside the SimpleRAG-Deployment-Demo directory when running docker build. Run ls (macOS) or dir (Windows) to confirm requirements.txt is present in the current folder.

Application starts but returns 500 errors

Symptom: App loads but requests fail with Internal Server Error

NOTE: This usually means the OPENAI_API_KEY is missing or invalid. Make sure you ran the container with --env-file .env and your .env file contains a valid key.